Assignment 1 Report
2D Diffusion on Orthogonal and Non-Orthogonal Grids

Ali Tayyar

March 18, 2026

1 Introduction

The goal of this assignment was to solve a steady two-dimensional diffusion problem using the finite volume method on a skewed non-orthogonal grid. I wrote the MATLAB code in small functions so we can track the errors, I started with the geometric part, then the physics part, the boundary conditions, and then I solved the linear solver, which are checked separately. This structure was especially useful once the non-orthogonal correction term started causing problems, because it let me isolate each part of the implementation instead of debugging everything at once.

My final workflow followed the same order in which I built the code. I first prepared the geometric functions, then added the physical properties and interpolation routines, then implemented the boundary conditions and the system assembly, and finally focused on the non-orthogonal correction. This order turned out to be nice, because the correction term depends on almost every geometric quantity that is computed earlier.

Note: Please note that I included all the functions in one file named `Allfunctions.m`. so you don't have to open many files.

2 Problem Statement

The governing equation solved in this work is the steady Poisson's equation for difussion:

$$-\nabla \cdot (\Gamma \nabla T) = S.$$

both the diffusion coefficient and the source term depend on temperature:

$$\Gamma(T, x, y) = \frac{T}{400}\left(x^2 + e^{0.1y}\right), \qquad S(T, x, y) = \frac{T}{400}\left(2x - 0.2y\right).$$

The boundary conditions used in the actual problem were:

- West boundary: Dirichlet, $T = 400$
- South boundary: Dirichlet, $T = 320$
- North boundary: Neumann, $\partial T/\partial n = 0$
- East boundary: Robin, with $h = 15$ and $T_\infty = 300$

The non-orthogonal grid was generated in `Grid.m` from the four boundary curves of the physical domain provided by Adam, thanks adam :). Due to the source term, it turns to be a nonlinear equation that will be solved using iterative process.

3 Code Development Order

I organized the implementation in the following order.

3.1 Geometric Functions

We started with the geometric functions because every later step depends on them. These functions are relatively simple, where we define the grid and the quantities required by the finite volume method before looking at anything.

- `Grid.m` and `Grid_test.m`
- `getCentroids.m`
- `getCellVolumes.m`
- `getSurfaceVectors.m`
- `getFaceCenters.m`
- `getCFVectors.m`
- `getInterpolationFactors.m`

At this stage, I did one plot using chat gpt to check what are things looking like, I relied on ChatGPT to debug the plots because it saves so much time, to check face normals, cell centers, and center-to-center vectors were consistent. We also used geometry plotting functions to verify the grid and vector directions visually.

Then I plotted th geometric part for a 10*10 mesh to check if everything is intact.

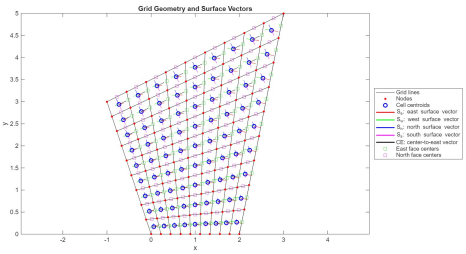

Figure 1: Geometry verification plot showing the mesh, centroids, face centers, and surface vectors.

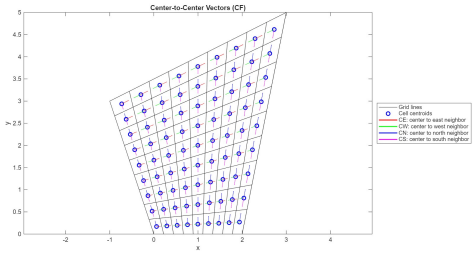

Figure 2: Center-to-center neighbor vectors used in the finite volume discretization.

3.2 2. Physics and Discretization Functions

Once the geometry was working, I moved to the physical part of the problem:

- `computeMaterialProperties.m`
- `interpolateGammaToFaces.m`
- `interpolateTemperatureToFaces.m`
- `computeCellGradient.m`
- `decomposeSurfaceVectors.m`
- `computeNonOrthogonalCorrection.m`
- `computeDiffusionCoefficients.m`

This part of the code contains the actual finite volume discretization. The interpolation factors were used for face values, the Green-Gauss theorem which is then used in the non-orthogonal term, and the surface vectors were decomposed into orthogonal and non-orthogonal parts.

3.3 3. Boundary Conditions and System Assembly

After that, we implemented the boundary treatment and the final algebraic system:

- `applyBoundaryFaceTemperatures.m`
- `applyBoundaryConditions.m`
- `applyAnalyticalBoundaryConditions.m`
- `assembleSystem.m`

The analytical boundary condition function was used only for the uniform-grid benchmark problem. The general boundary condition function was used for the actual skewed-grid case.

3.4 4. Linear Solver and Driver Scripts

Finally, we connected everything in the driver scripts:

- `solveLinearSystem.m`
- `solveTDMA.m`
- `solveGaussSeidel.m`
- `Main_TEST.m`, for the uniform grid
- `Main.m`

For the results, the choices can be chosen between:

- Interpolation factor method: `distance, volume`
- Surface-vector decomposition: `overrelaxed, minimum, normal`
- Linear solver: `TDMA, gauss seidel`

where the code can choose between them and the code can handle them all.

4 Uniform-Grid Benchmark and Analytical Check

Before trusting the code on the non-orthogonal mesh, I created a separate benchmark problem on a uniform grid using `Grid_test.m`. This benchmark was very useful because it gave me a case with a known analytical solution, so I could check the implementation against an exact reference rather than only relying on visual inspection.

For the benchmark case, we used:

- Uniform orthogonal grid
- Constant diffusion coefficient, $\Gamma = 1$
- Zero source term, $S = 0$
- West boundary: $T = 400$
- East boundary: $T = 300$
- North and south boundaries: zero normal gradient

The boundaries used here are in a function called `applyAnalyticalBoundaryConditions.m` Under these conditions, the exact solution is a linear temperature distribution in the horizontal direction:

$$T_{\text{exact}} = 400 - 100\hat{x}, \qquad \hat{x} = \frac{x - x_{\min}}{x_{\max} - x_{\min}}.$$

This benchmark helped me confirm that the diffusion discretization, the interpolation, and the assembly process were correct before moving back to the skewed grid.

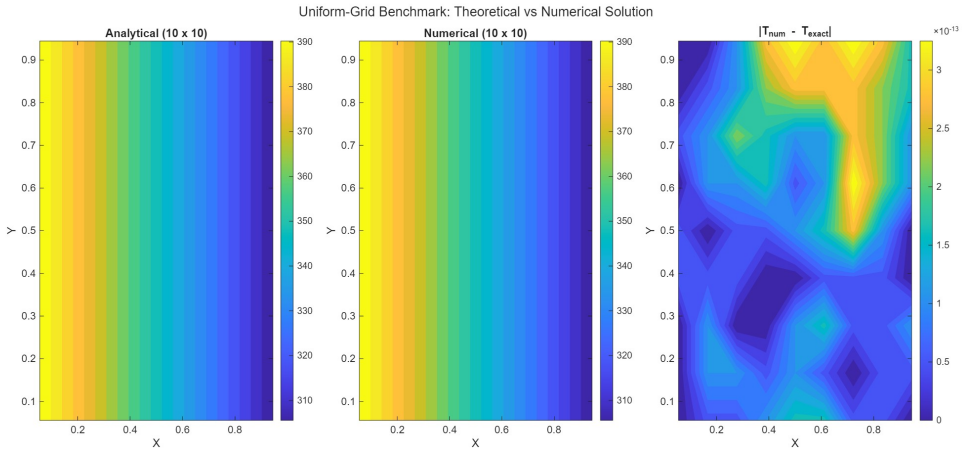

Figure 3: Uniform-grid benchmark result compared with the analytical solution.

Table 1: Uniform-grid benchmark errors. Replace the entries below with the exact values printed by `Main_TEST.m`.

m	n	L_2 error	Max error
10	10	1.492e-13	3.4106e-13
20	20	4.4213e-13	1.4211e-12
40	40	2.8884e-12	7.5602e-12
80	80	1.5273e-11	2.5125e-11

5 Main Difficulty: The `bcorr` Term

The main difficulty in this assignment was the non-orthogonal correction term, stored in the code as `bcorr`. This was the part that took the most debugging time. A useful observation was that the code behaved reasonably when `bcorr` was set to zero, which told me that the main diffusion coefficients, the boundary conditions, and the linear solver were already working. The instability appeared only when the correction term was activated. Also, I checked for the Boundary conditions to check whether the problem is there, so I started with simple BCs and then moved to this multi-usable BC function.

This is exactly why the uniform-grid benchmark became important. On an orthogonal grid, the non-orthogonal correction should vanish, so the benchmark gave us a clean way to separate the base discretization from the correction term. Once the benchmark worked, I knew the main issue had to be in the way the correction was being fed by gradients and boundary-face values.

In the final implementation, I made sure that boundary face temperatures were treated consistently before computing the Green-Gauss gradient, and we evaluated the correction term using face-based gradients rather than only reusing the owner-cell gradient directly. After that change, the solution became stable with `bcorr` activated, and the multiple-mesh runs gave consistent temperature ranges and small residuals.

6 Results on the Non-Orthogonal Grid

After the benchmark check, I ran the actual problem on the non-orthogonal grid using several mesh sizes. All results reported in Table 2 correspond to the same numerical choices:

- interpolation factor method: `distance`
- decomposition method: `overrelaxed`

- linear solver: `TDMA`

Table 2: Results for the non-orthogonal grid using distance interpolation, overrelaxed decomposition, and TDMA.

m	n	Outer iterations	Max residual	$T_{\min}$	$T_{\max}$
20	20	18	2.651×10^{-10}	301.22	396.24
40	40	20	2.7567×10^{-10}	301.00	398.24
80	80	21	2.9427×10^{-10}	300.89	399.15
160	160	23	4.8881×10^{-10}	300.83	399.58

These results show that the solution remains stable as the mesh is refined. The number of outer iterations increases only slightly, from 18 to 23 iterations, while the residual stays very small for all meshes. The temperature range also approaches a stable interval, with the minimum temperature moving closer to 300 and the maximum moving closer to 400 as the grid becomes finer.

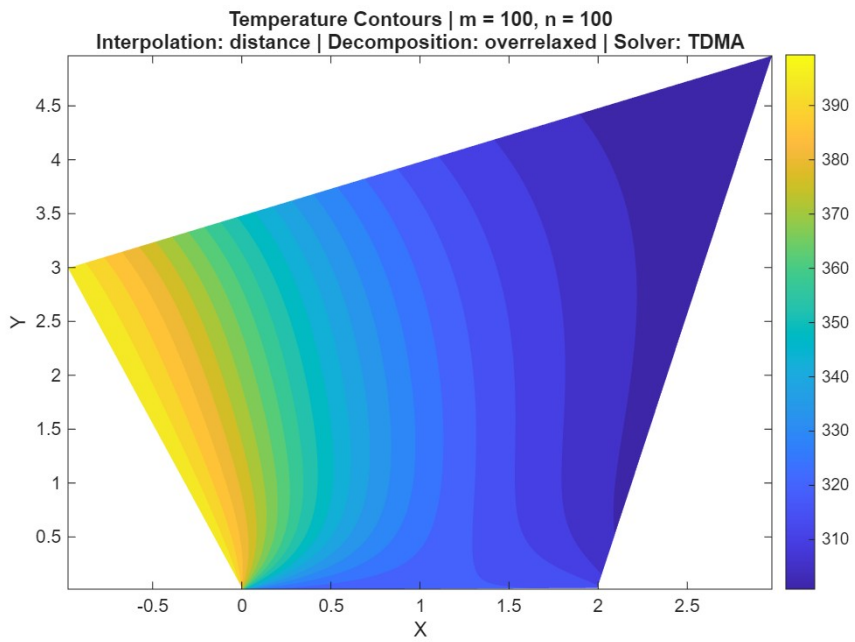

Figure 4: Temperature contours on the non-orthogonal grid for one representative mesh.

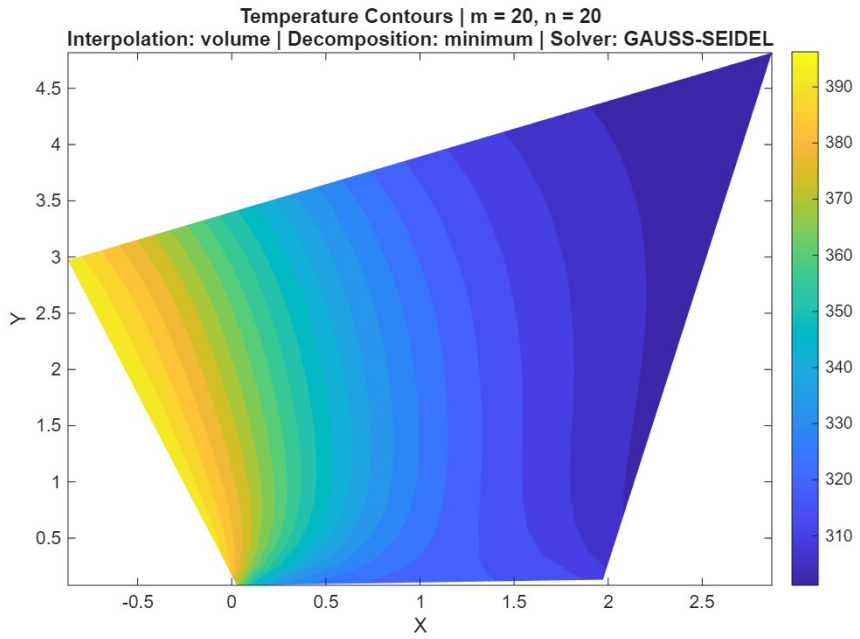

Figure 5: Temperature plots for a different mesh size. with different interpolation scheme, solver, and decomposition method

7 Discussion

Looking back, the most important lesson from this assignment was that non-orthogonal corrections cannot be trusted unless the geometric part and the benchmark case are already solid. It was tempting at first to focus immediately on the final physical case, but the simpler orthogonal test problem gave me a much clearer debugging path.

Another important point is that organizing the code into small functions made the project much easier to manage. When the correction term failed, we did not have to search through one long script. Instead, we could separately inspect the geometry, interpolation, boundary treatment, gradient calculation, and system assembly.

The final code structure also made mesh studies easier to perform, since the same workflow could be repeated for several (m, n) pairs without rewriting the main script.

7.1 GitHub Repository: ComputationalFluidMechanics

In this assignment, we developed a MATLAB finite volume solver for a two-dimensional steady diffusion problem on a non-orthogonal grid. I first built and verified the geometric functions, then added the physical properties and discretization, then implemented the boundary conditions and the linear solver. A separate uniform-grid benchmark with an

analytical solution was used to validate the implementation before studying the actual skewed-grid case.

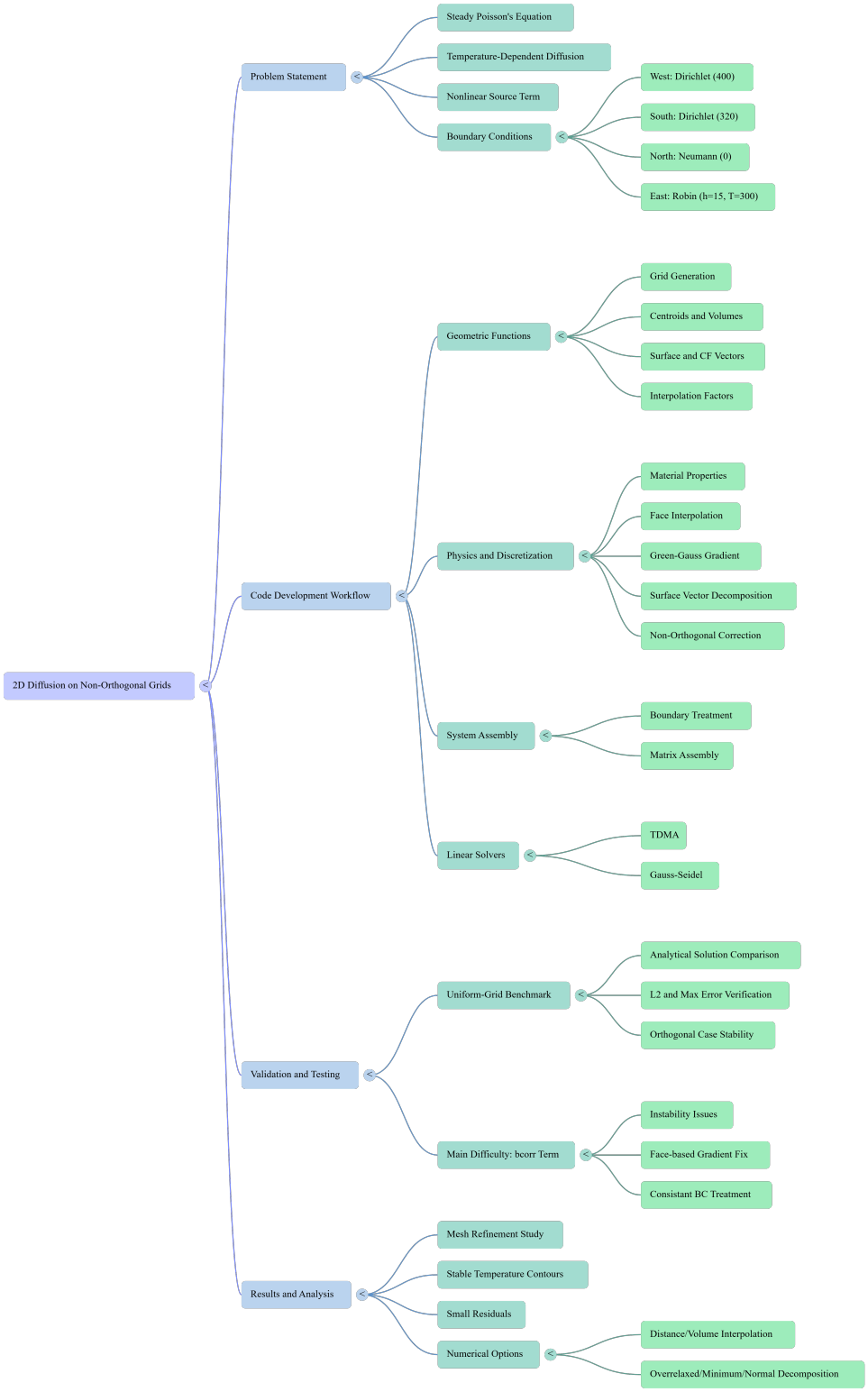

Figure 6: Conceptual workflow of the project, showing the problem setup, code-development stages, validation strategy, and the main difficulty associated with the non-orthogonal correction term.